

LLM 서비스 개발을 위한
프레임워크, LangChain

LLM 서비스 개발을 위한 프레임워크, LangChain

1. LangChain이란 무엇인가!
2. 체인을 구현하는 문법, LCEL

1. LangChain이란 무엇인가!

- 최근 LLM 기술의 비약적인 발전과는 별개로, 이를 활용한 실질적인 비즈니스 가치를 증명(PoV)하는 서비스를 구현하는 것은 여전히 어려운 과제임
- 랭체인 프레임워크
 - LLM 기반 AI 에이전트 서비스를 개발할 수 있게 해주는 도구
 - 기존 LLM 모델이 단순히 질문에 답변하는 수준이었다면, 이제는 기업이 축적한 데이터와 구축한 서비스를 AI가 활용하는 수준으로 향상시킬 수 있게 되었음
 - LLM 기반 서비스 구현에 있어 더 효율적이고 쉽게 개발할 수 있도록 도와주는 도구

1. LangChain이란 무엇인가!

- LLM 기반 서비스, 챗GPT
 - 검색 기반 답변 생성 및 출처 제공
 - 지식 기반 답변 생성
 - 과거 대화 맥락 기반 답변 생성
 - 데이터 시각화
 - 외부 도구 연동

1. LangChain이란 무엇인가!

- 랭체인 작동 원리: ‘체인’

- 랭체인: ‘언어(language) 모델’의 ‘Language’와 ‘Chain’의 합성어
- 여기서 ‘체인(chain)’은 랭체인이 작동하는 핵심 방식이자 개념을 담고 있음
- 사슬이 엮여 있는 것처럼 LLM 서비스 개발에 필요한 다양한 기능을 하나로 엮는 것
- 이 체인들이 연달아 작동하면서 더욱 정교하고 고도화된 업무를 수행

- 사용자의 질문에 단순히 답변만 생성하는 간단한 챗봇을 개발한다면 오픈AI에서 제공하는 LLM 모델을 API로 호출하면 구현 가능

- 하지만, 고도화된 LLM 기반 서비스를 개발하려면 단순히 오픈AI API만 사용해서는 개발 난이도가 굉장히 높아짐

1. LangChain이란 무엇인가!

- 랭체인 학습을 위한 참고 자료

- 랭체인 공식 문서 - 최신 버전 확인 등
 - <https://docs.langchain.com/oss/python/langchain/overview>
- 랭체인 공식 깃허브 - 특정 메서드나 클래스 정보
 - <https://github.com/langchain-ai/langchain>
- 랭체인 공식 블로그 - 최신 정보, 아이디어, 인사이트를 얻는데 유용
 - <https://blog.langchain.com/>
- 랭체인 공식 간행물 - 최신 동향이나 통계 자료를 공유
 - <https://blog.langchain.com/langchain-state-of-ai-2024>

2. 체인을 구현하는 문법, LCEL

- (실습) 실습 파일 준비(LCEL.ipynb)

- 가상 환경 설정

```
# 가상 환경 생성
$ python -m venv .venv

# 윈도우 가상 환경 활성화
$ source .venv/Scripts/activate # 맥 환경에서는 Scripts를 bin으로 변경(.venv)
$ python -m pip install --upgrade pip
```

- 필요 패키지 설치

```
(.venv)
$ pip install langchain langchain-openai
```

- 버전 확인

```
(.venv)
$ pip show langchain
$ pip show langchain-openai
$ pip show langchain-core
```

```
Name: langchain
Version: 1.2.14
... ( 중략 ) ...
Name: langchain-openai
Version: 1.1.12
... ( 중략 ) ...
Name: langchain-core
Version: 1.2.25
... ( 후략 ) ...
```

2. 체인을 구현하는 문법, LCEL

- LCEL(LangChain Expression Language) 문법 이해

1. LCEL 체인의 구축 및 실행

- LCEL의 세 가지 핵심 요소
 - 프롬프트(Prompt): LLM에 전달할 지시 또는 질문
 - 모델(Model): LLM 모델(예: 오픈AI의 gpt-5-nano)
 - 출력 파서(Output Parser): 모델이 생성한 응답을 원하는 형식(예: 문자열)으로 변환
- 이렇게 정의한 세 가지 요소를 Unix 파이프 연산자(|)로 연결하면 하나의 체인이 만들어짐

Prompt | Model | Output Parser

- 체인을 구성하는 세 가지 요소

2. 체인을 구현하는 문법, LCEL

- LCEL(LangChain Expression Language) 문법 이해

1. LCEL 체인의 구축 및 실행

- 프롬프트 정의

```
from langchain_core.prompts import PromptTemplate

prompt = PromptTemplate.from_template("{topic}에 대해 쉽게 설명해주세요.")
```

- from_template() 메서드로 프롬프트 정의하기
- 중괄호({}) 안에 변수명을 작성하여 동적인 입력 변수를 정의하여, 다양한 값을 동적으로 전달해 프롬프트를 완성할 수 있음

- LLM 모델 객체 생성

```
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model="gpt-5-nano")
```

- 오픈 AI의 LLM 모델 불러오기
- API Key는 위에서 불러왔으므로 별도 과정 필요 없음

2. 체인을 구현하는 문법, LCEL

- LCEL(LangChain Expression Language) 문법 이해

1. LCEL 체인의 구축 및 실행

- 출력 파서 정의

```
from langchain_core.output_parsers import StrOutputParser

output_parser = StrOutputParser()
```

- LLM 모델이 생성한 응답을 적절한 형태로 변환함
- 가장 기본적 파서인 모델의 응답을 문자열(string)로 변환하는 StrOutputParser를 사용

- 구성 요소 연결

```
chain = prompt | model | output_parser
```

- 파이프 연산자(|)로 연결해 하나의 체인 구축

2. 체인을 구현하는 문법, LCEL

- LCEL(LangChain Expression Language) 문법 이해

1. LCEL 체인의 구축 및 실행

- 최종 출력 생성

```
response = chain.invoke({"topic": "양자역학"})  
print(response)
```

아주 쉽게 말해요. 양자역학은 아주 작은 세계(원자나 그보다 작은 입자)에서 물질과 빛이 작동하는 규칙을 다루는 이론이에요. 우리의 매일 세계의 규칙과는 다른 점이 많아서 처음엔 이상하게 느껴질 수 있어요. 주요 아이디어를 간단히 정리해볼게요.- 에너지는 덩어리로 존재한다(양자화): 어떤 현상은 연속적으로 바뀌는 게 아니라 특정한 '양'으로만 바뀌어요. 예를 들어 원자는 특정 에너지 레벨만 가지거나 빛의 에너지도 특정 값들로만 흘러가요.- 입자도 파동처럼 행동하고, 파동도 입자처럼 행동한다(이중성): 빛이나 전자 같은 입자는 실체처럼 보이기도 하지만 간섭처럼 파동의 성질도 보여줘요. 예를 들어 한 입자도 두 슬릿을 동시에 지나가는 것처럼 보일 수 있어요.- 가능성의 중첩: 작은 세계의 물체는 측정되기 전에 여러 상태가 동시에 '가능한 상태'로 함께 존재해요. 뭔가를 측정하면 그 중 하나로 '확정'되는데, 왜 그 하나가 나오는지 확률로만 설명돼요.- 확률로 기술된다: 한 번의 실험으로 특정 결과가 확실하게 예측되지는 않아요. 같은 조건에서 여러 번 실험하면 어떤 결과가 얼마나 자주 나오는지 확률적으로 예측할 수 있어요. 이 확률은 보통 수학적으로 파동처럼 퍼지는 형태로 나타난다고 말합니다.- 불확정성: 특정 쌍의 물리량은 동시에 정확히 알기 어렵거나 불가능해요. 예를 들어 위치와 속도처럼 한 쌍은 정밀하게 알면 다른 하나는 많이 불확실해져요. 이게 바로 '불확정성 원리'예요.- 얽힘과 비국소성: 서로 멀리 떨어진 두 입자가 아주 강하게 연결되어 있어 한 입자를 측정하면 다른 입자의 상태도 즉시 결정되는 것처럼 보일 수 있어요. 이건 공간이 멀리 떨어져 있어도 영향을 주고받는다라는 양자 특성의 한 예예요.- 측정은 상태를 바꾼다: 관찰하거나 측정하는 행위 자체가 시스템의 순서를 바꿔 버려요. 그래서 고전 세계처럼 "있던 것을 그냥 알아보는 것"이 아니라, 측정 순간에 현실이 확정된다는 느낌이 생겨요. 환경과의 상호작용으로도 비슷하게 고전적 세계가 드러나게 돼요(디코히런스).- 실생활과의 연결: 이 이론이 없었다면 트랜지스터(컴퓨터의 뇌를 이루는 부품), 레이저, 자가 진단용 의료기기 MRI, 양자 컴퓨팅의 원리 같은 현대 기술이 나올 수 없었을 거예요. 간단한 비유로 이해하기- 두 슬릿 실험: 입자가 파동처럼 간섭 패턴을 만들어 내다가, 실제로 측정해서 어느 슬릿을 지나갔는지 확인하면 입자 하나의 흔적이 남아요. 안 보일 때만 파동처럼 여러 가능성이 겹쳐 있는 상태가 되는 거죠.- 중첩과 측정: 예를 들어 스핀이라는 물리량은 위쪽일 수도 아래쪽일 수도 두 가능성이 동시에 존재하다가, 측정하면 하나의 방향으로 '확정'돼요. 필요하다면 특정 부분을 더 자세히 설명해 드릴게요- 예를 들어 double-slit 실험 이야기나, 얽힘의 간단한 예시, 혹은 양자 컴퓨팅의 아이디어 중 무엇이 궁금하신가요?

2. 체인을 구현하는 문법, LCEL

- LCEL(LangChain Expression Language) 문법 이해

1. LCEL 체인의 구축 및 실행

- Invoke 메서드로 체인이 실행될 때 진행되는 절차

- 1) invoke 메서드에 입력된 딕셔너리({"topic": "양자역학"})가 프롬프트로 전달되어 완전한 프롬프트(양자역학에 대해 쉽게 설명해 주세요.)가 완성됨
- 2) 완성된 프롬프트가 LLM 모델로 전달되어 LLM이 응답을 생성함
- 3) LLM이 생성한 답변이 출력 파서로 전달되어 우리가 원하는 형태로 파싱됨
- 4) 최종 결과가 문자열로 반환됨

2. 체인을 구현하는 문법, LCEL

- LCEL(LangChain Expression Language) 문법 이해

1. LCEL 체인의 구축 및 실행

- Stream 메서드 실행

```
for chunk in chain.stream({"topic": "양자역학"}):  
    print(chunk, end='')
```

- 챗GPT처럼 답변이 실시간으로 생성되는 듯 자연스러운 연출 구현
- 답변을 작은 말뭉치(또는 청크(chunk))로 나누어 반환함
- 이 말뭉치들을 파이썬 반복문을 사용해 실시간 스트리밍처럼 보이도록 출력

2. 체인을 구현하는 문법, LCEL

- LCEL(LangChain Expression Language) 문법 이해

1. LCEL 체인의 구축 및 실행

- 프롬프트에 다수의 변수 정의하기
 - 2개의 입력 변수를 정의하고, invoke 메서드에 전달하려면?

```
# 새로운 프롬프트 정의
prompt2 = PromptTemplate.from_template("{topic}에 대해 {level}수준으로 설명해 주세요.")

# 새로운 체인 생성
chain2 = prompt2 | model | output_parser

# 체인 실행
response = chain2.invoke({"topic": "인공지능", "level": "초등학생"})
print(response)
```

2. 체인을 구현하는 문법, LCEL

- LCEL(LangChain Expression Language) 문법 이해

2. LCEL 구축 시 주의 사항

- 다중 입력 변수 처리 및 KeyError
 - invoke 메서드 호출 시 프롬프트에 정의한 입력 변수에 해당하는 값을 반드시 키-값 쌍 모두 딕셔너리로 전달해야 함
 - 앞서 두 개의 입력 변수 `topic`과 `level`을 프롬프트에 정의한 경우, `invoke` 메서드를 호출할 때 하나라도 빠트리면 `KeyError`가 발생함

```
# 잘못된 호출 (level 변수 누락 -> KeyError 발생)
response_error = chain2.invoke({"topic": "양자역학"}) # KeyError 발생!
```

- 체인 구성 순서 및 ValueError
 - 체인은 반드시 프롬프트(prompt) -> 모델(model) -> 출력 파서(output_parser) 순서로 연결되어야 함

```
# 잘못된 체인 구성
chain2 = model | prompt2 | output_parser
chain2.invoke({"topic": "인공지능", "level" : "초등학생"}) # ValueError 발생!
```

2. 체인을 구현하는 문법, LCEL

- LCEL(LangChain Expression Language) 문법 이해

3. 결합된 체인 구축

- LCEL을 사용하면 한 번의 invoke 메서드로 여러 체인을 순서대로 실행할 수 있음
- 이를 결합된 체인(combined chain)이라고 함
- 이 기능은 복잡한 워크플로우를 구현하거나, 이전 체인의 결과를 다음 체인의 입력으로 활용하여 LLM 서비스의 품질을 고도화할 때 유용
- 결합된 체인의 작동 원리
 - 첫 번째 체인 실행
 - 첫 번째 체인 결과 생성
 - 두 번째 체인으로 딕셔너리 전달
 - 두 번째 체인 실행

2. 체인을 구현하는 문법, LCEL

- LCEL(LangChain Expression Language) 문법 이해

3. 결합된 체인 구축

- 첫 번째 체인 생성

```
# 첫 번째 체인의 프롬프트
prompt1 = PromptTemplate.from_template("{topic}에 대해 {level} 수준으로 설명해 주세요.")

# 첫 번째 체인 생성
chain1 = prompt1 | model | output_parser

# 새로운 딕셔너리 생성
chain1_result_dict = {"chain1_result": chain1}
```

- 두 번째 체인 생성

```
# 두 번째 체인의 프롬프트
prompt2 = PromptTemplate.from_template("이 정보가 유익한가요? 정보 : {chain1_result}")

# combined_chain 생성
combined_chain = chain1_result_dict | prompt2 | model | output_parser
```

2. 체인을 구현하는 문법, LCEL

- LCEL(LangChain Expression Language) 문법 이해

3. 결합된 체인 구축

- 결합된 체인 실행

```
final_result = combined_chain.invoke({"topic": "인공지능", "level": "초등학생"})  
print(final_result)
```

“””

네, 이 정보는 초보자에게 아주 유익하고 이해하기 쉬운 기본 설명이에요. 핵심 아이디어를 잘 전달하고 있고, 일상 사례와 안전 수칙도 잘 담겨 있습니다. 장점- AI가 하는 일의 큰 그림을 쉽게 이해하게 설명해요: 데이터를 통해 패턴을 배우고, 그 규칙으로 문제를 해결하는 방식.- 일상에서의 활용 예시가 구체적이라 흥미를 끌어요.- 사람과의 차이와 안전하게 쓰는 법이 함께 있어 안전한 사용 습관을 강조합니다.- 비유가 친근하고, 추가로 궁금한 주제를 함께 생각해 볼 수 있도록 제안합니다. 다듬으면 좋을 부분(간단한 보완 제안)- AI의 다양성에 대한 언급 확장: 실제로는 규칙 기반 시스템도 있고, 딥러닝처럼 데이터에서 배우는 방식도 있다는 점을 짚어 주면 더 정확해요.- “AI는 스스로 느끼거나 생각한 다”는 표현은 다소 단순화될 수 있어요. 모델이 목표를 가지려면 사람이나 시스템이 설정한 목표가 필요하다는 점을 덧붙이면 좋습니다.- 길 찾기 예시를 더 구체화하면 더 명확해져요. 예를 들어 지도 데이터, 실시간 교통 정보, 경로 계산 알고리즘(A*나 다익스트라 같은) 등이 함께 작동한다는 점을 간단히 덧붙이기.- 안전 수칙에 “생성된 내용의 신뢰도 확인” 같은 포인트를 하나 더 추가하면 좋아요. 예: AI가 만든 정보는 반드시 사람의 확인이 필요할 때가 많다.- 좀 더 다양한 대상에 맞춘 버전 제안: 아이들용, 학부모/교사용, 일반 성인용 등으로 버전을 다르게 준비하면 활용도가 올라갑니다. 간단한 보완 예시 문장(원문에 추가하면 좋을 내용)- AI의 다양성: “AI에는 컴퓨터가 규칙을 배워서 문제를 푸는 방식도 있고, 사람이 만든 규칙을 이용하는 방식도 있어요.”- 길 찾기 내역 보완: “지도 데이터와 실시간 교통 정보, 그리고 경로를 계산하는 알고리즘이 함께 작동해 가장 좋은 길을 제시해요.”- 신뢰도와 확인: “AI가 남긴 답을 그대로 믿지 말고, 중요한 결정은 사람의 확인이 필요해요.” 다음에 도와줄 수 있는 것- 원하시면 이 내용을 아이들용으로 더 간단하게 바꾸거나, 선생님/부모님용으로 설명 자료로 쓰기 좋게 다듬어 드릴게요.- “AI가 어떻게 길을 찾을까?”, “AI가 잘못하면 어떻게 고칠까?” 같은 구체적인 질문에 대한 간단한 설명도 함께 정리해 드릴 수 있습니다. 혹시 더 구체적으로 어떤 대상자(아이·부모님·교사)나 톤으로 다듬고 싶으신가요? 원하시면 바로 맞춤형 버전으로 만들어 드릴게요.

“””

2. 체인을 구현하는 문법, LCEL

- LCEL(LangChain Expression Language) 문법 이해

3. 결합된 체인 구축

- 결합된 체인 구축 시

- 단 한 번의 invoke 메서드로 여러 개의 체인을 순서대로 실행 시킬 수 있음
- 이를 통해, 한 번의 단순한 질문과 응답을 벗어나 더욱 복잡하고, 체계적인 답변을 얻어낼 수 있음
- 프롬프트가 과도하게 복잡해지는 것을 방지할 수 있음
- 필요하다면 각각의 체인을 독립적으로 활용할 수 있음
- 결합된 체인을 구축할 때는 특히 KeyError에 유의
- 후속 체인에게 전달되어야 할 입력 변수가 올바르게 전달되면 기껏 구축한 결합된 체인이 실행조차 되지 못하게 됨
- 다수의 체인을 결합하고 온전히 사용하기 위해서는 과정에 대한 설계가 매우 중요

2. 체인을 구현하는 문법, LCEL

- LCEL(LangChain Expression Language) 문법 이해

- 4. 프롬프트

- AI 모델에게 내리는 지시 또는 명령을 의미
 - 프롬프트를 랭체인 문법에 맞춰 작성하는 방법 3가지
 - 1) PromptTemplate.from_template()
 - 2) PromptTemplate
 - 3) ChatPromptTemplate

2. 체인을 구현하는 문법, LCEL

- LCEL(LangChain Expression Language) 문법 이해

4. 프롬프트

1) PromptTemplate.from_template()

- 지금까지 사용해 왔던 방법
- 가장 단순하며 다양한 상황에서 융통성 있게 활용 가능한 방법

```
from langchain_core.prompts import PromptTemplate

prompt = PromptTemplate.from_template("{topic}에 대해 {level} 수준으로 설명해주세요.")
print(prompt)
```

```
input_variables=['level', 'topic'] input_types={} partial_variables={} template='{topic}에 대해 {level} 수준으로 설명해주세요.'
```

2. 체인을 구현하는 문법, LCEL

- LCEL(LangChain Expression Language) 문법 이해

4. 프롬프트

2) PromptTemplate

- from_template 메서드 없이 PromptTemplate를 직접 사용하는 방법

```
from langchain_core.prompts import PromptTemplate

prompt = PromptTemplate(
    template= "{topic}에 대해 {level} 수준으로 설명해 주세요",
    input_variables=["topic", "level"],
    partial_variables={
        "topic": "양자역학"
    }
)
print(prompt)
```

- PromptTemplate 필수 인자
 - template : LLM 모델에게 전달하는 프롬프트 문자열
 - input_variables : template에 작성한 입력 변수({topic}, {level})를 리스트 형태로 작성
 - partial_variables : 특정 변수의 값을 미리 지정할 때 사용

2. 체인을 구현하는 문법, LCEL

- LCEL(LangChain Expression Language) 문법 이해

4. 프롬프트

2) PromptTemplate

- input_variables와 partial_variables 확인

```
print(prompt.input_variables)    # ['level']  
print(prompt.partial_variables)  # {'topic': '양자역학'}
```

- partial_variables

- 특정 변수의 값을 사전에 설정하기 위해 사용

```
from langchain_core.prompts import PromptTemplate  
  
prompt = PromptTemplate(  
    template= "{year} + {n} + {m}는 무엇인가요?",  
    partial_variables={  
        "year": "2026"  
    }  
)  
chain = prompt | model | output_parser  
chain.invoke({"n": "3", "m": "5"}) # 답변 예시 : '2033입니다. 계산: 2026 + 3 = 2029, 2029 + 5 = 2034.'
```

2. 체인을 구현하는 문법, LCEL

- LCEL(LangChain Expression Language) 문법 이해

4. 프롬프트

2) PromptTemplate

- partial_variables

- 함수 연결도 가능

```
from datetime import datetime

# 올해 연도를 문자열로 반환하는 함수
def get_current_year():
    return str(datetime.now().year)

prompt = PromptTemplate(
    template= "{year} + {n} + {m}는 무엇인가요?",
    partial_variables={"year": get_current_year},
)
chain = prompt | model | output_parser
chain.invoke({"n": "3", "m": "5"})
```


2. 체인을 구현하는 문법, LCEL

- LCEL(LangChain Expression Language) 문법 이해

4. 프롬프트

2) PromptTemplate

- `partial_variables` 주의 사항
 - `partial_variables`에 'n'을 10으로 미리 할당했더라도, 'invoke' 메서드 호출 시 새로운 'n' 값을 전달하면 메서드 호출 시 전달된 값으로 업데이트되어 프롬프트가 완성

```
prompt = PromptTemplate(  
    template= "{year} + {n} + {m}는 무엇인가요?",  
    partial_variables={  
        "year": "2026",  
        "n": "10"  
    },  
)  
  
chain = prompt | model | output_parser  
chain.invoke({"n": "3", "m": "5"})
```

2. 체인을 구현하는 문법, LCEL

- LCEL(LangChain Expression Language) 문법 이해

4. 프롬프트

3) ChatPromptTemplate

- 이전 대화 내역을 추가함으로써 과거 대화와 연관된 답변을 생성하게 할 수 있게 함

```
from langchain_core.prompts import ChatPromptTemplate

prompt = ChatPromptTemplate([
    ("system", "당신은 강아지 안구 건강 관련 조언을 해주는 AI 챗봇입니다."),
    ("human", "안녕하세요! 저희 집 강아지가 눈을 자주 긁어요"),
    ("ai", "그렇군요. 사용자님의 강아지는 몇 살인가요?"),
    ("human", "{user_input}"),
])

chain = prompt | model | output_parser
chain.invoke({"user_input" : "3살이에요"})
```

2. 체인을 구현하는 문법, LCEL

- LCEL(LangChain Expression Language) 문법 이해

5. 모델

- 랭체인은 다양한 LLM 제공자(provider)들을 통합 지원
 - anthropic 모델 사용 예시

```
pip install -U langchain-anthropic
```

```
# from langchain_openai import ChatOpenAI
from langchain_anthropic import ChatAnthropic

# claude 모델을 위한 API Key를 발급 받지 않았으므로,
# 이번 실습에서는 사용하지 않습니다.
# model = ChatOpenAI(model="gpt-5-nano")
model = ChatAnthropic(model="claude-3-sonnet")
```

2. 체인을 구현하는 문법, LCEL

- LCEL(LangChain Expression Language) 문법 이해

5. 모델

- 오픈 AI 모델 하이퍼파라미터

```
from langchain_openai import ChatOpenAI

model = ChatOpenAI(
    model="gpt-5-nano",
    temperature=0.8,
    max_tokens=2048,
    max_retries=2
)
```

- model: 오픈AI에서 제공하는 여러 LLM 모델 중 특정 모델을 지정
- temperature: 생성되는 텍스트의 다양성을 조절하는 매개변수
 - 낮은 값(예: 0.2)은 더 결정적이고 일관된 출력을 생성
 - 높은 값(예: 0.8)은 더 창의적이고 다양성이 높은 출력을 생성
 - 이 값이 높을수록 모델이 더 다양한 단어와 구문을 선택하도록 유도하여 창의적인 결과를 도출
- max_tokens: 모델이 생성할 수 있는 최대 토큰 수를 지정
- max_retries: 모델 호출이 실패할 경우 재시도 횟수를 지정

2. 체인을 구현하는 문법, LCEL

● LCEL(LangChain Expression Language) 문법 이해

6. 출력 파서(output parser)

- LLM 모델이 생성한 답변을 적절한 형태로 파싱(parsing), 즉 구조화할 때 사용하는 컴포넌트
- 체인을 이루는 요소 중 출력 파서는 필수 요소는 아님
- 체인은 프롬프트와 모델만으로도 구현 가능

```
from langchain_core.prompts import PromptTemplate
from langchain_openai import ChatOpenAI

prompt = PromptTemplate.from_template("{topic} 에 대해 쉽게 설명해 주세요.")
model = ChatOpenAI(
    model="gpt-5-nano",
    max_tokens=2048
)

chain = prompt | model
chain.invoke({"topic" : "인공지능"})
```

AIMessage(content='쉽게 말해 드릴게요.\n\n- AI(인공지능)란? 컴퓨터가 많은 데이터를 보고 규칙이나 패턴을 배워서, 우리가 주는 문제에 대해 자동으로 해결해 주는 기술 이에요. 사람처럼 생각한대기보다, 데이터에서 배운 규칙에 따라 판단하거나 예측을 하는 거예요.\n\n- 어떻게 작동하나요? 아주 간단한 흐름은 이렇습니다.\n\n1) 데이터: 예를 들어 이메일, 사진, 글 등 많은 예시를 모아요.\n\n2) 알고리즘: 데이터를 분석해 규칙을 찾는 방법(수학적 규칙)을 정합니다.\n\n3) 모델: 알고리즘이 데이터를 바탕으로 ...
(생략) ...

'model_provider': 'openai', 'model_name': 'gpt-5-nano-2025-08-07', 'system_fingerprint': None, 'id': 'chatcmpl-DQV8NPNomW02bhvii2RkbOAis1uSG', 'service_tier': 'default', 'finish_reason': 'stop', 'logprobs': None, id='lc_run--019d52ad-39e2-7573-987b-ca82f6f9281a-0', tool_calls=[], invalid_tool_calls=[], usage_metadata={'input_tokens': 16, 'output_tokens': 1751, 'total_tokens': 1767, 'input_token_details': {'audio': 0, 'cache_read': 0}, 'output_token_details': {'audio': 0, 'reasoning': 1088}})

2. 체인을 구현하는 문법, LCEL

- LCEL(LangChain Expression Language) 문법 이해

- 6. 출력 파서(output parser)

- 체인에 출력 파서 연결 시 장점
 - 가독성 향상 :
 - 출력 파서를 연결하면 AIMessage 객체가 가진 content 속성의 문자열 값만 추출해 출력할 수 있음
 - 활용성 향상
 - 결합된 체인(combined_chain)과 같이 복잡한 파이프라인을 구축할 때, 첫 번째 체인의 실행 결과를 두 번째 체인에게 넘길 때, AIMessage 객체 자체를 넘기는 것은 비효율적
 - 불가능한 것은 아니지만 LLM에게 불필요한 내용까지 전달하게 됨으로써 우리가 사용 가능한 입력 토큰을 낭비
 - 출력 파서를 사용하면 이전 실습처럼 content의 값만 도려내어 반환 받을 수 있을 뿐만 아니라, 답변을 자신이 원하는 형태로 구조화하여 얻어낼 수도 있음
 - 랭체인은 Str, JSON, XML, CSV 등 다양한 형태의 데이터 포맷으로 파싱할 수 있는 여러 파서를 지원함

2. 체인을 구현하는 문법, LCEL

- LCEL(LangChain Expression Language) 문법 이해

- 6. 출력 파서(output parser)

- CommaSeparatedOutputParser, JsonOutputParser 등이 사용 빈도가 높음
 - 앞서 사용한 StrOutputParser와 달리 CommaSeparatedOutputParser, JsonOutputParser 파서들은 LLM 모델이 생성할 답변의 형태를 사전에 정의하여야 함
 - LLM 모델이 지정된 형태로 답변을 생성해야, 각 파서가 content의 값을 구조화할 수 있기 때문
 - 예를 들면
 - JSON 형태로 변환하고자 한다면, 모델이 생성해야 할 JSON의 구조를 프롬프트로 먼저 지시해야 함
 - 그러면 모델은 JSON 형태의 답변을 생성하고, 그 답변을 AIMessage의 content에 담아 응답
 - 출력 파서는 content에 담긴 문자열 데이터를 JSON 객체로 변환해 최종 결과를 반환
 - 주의할 점: 앞서 모델이 생성한 JSON 형태의 답변은 JSON 객체는 아님 -> 프롬프트로 인해 JSON 형태를 갖추도록 지시했지만, JSON 객체만 답변하는 것은 아니기 때문에 프롬프트를 잘 작성하는 것이 중요

2. 체인을 구현하는 문법, LCEL

- LCEL(LangChain Expression Language) 문법 이해

- 6. 출력 파서(output parser)

- 프롬프트 구성 예시 - `partial_variables`를 사용하여 구성

```
template = """
당신은 K-Pop 정보를 제공하는 AI입니다. 그룹의 멤버 이름을 말해주세요.
K-pop group : {name}

FORMAT :
{format}
"""

prompt = PromptTemplate(
    template=template,
    partial_variables={
        "format": "JSON 형태로 답변하십시오"
    },
)
```


2. 체인을 구현하는 문법, LCEL

- LCEL(LangChain Expression Language) 문법 이해

6. 출력 파서(output parser)

- 출력 파서마다 정확한 객체 정보를 get_format_instruction 메서드를 통해 얻을 수도 있음

```
from langchain_core.output_parsers import JsonOutputParser

output_parser = JsonOutputParser()
format_instructions = output_parser.get_format_instructions()

print(format_instructions) # Return a JSON object.

prompt = PromptTemplate(
    template=template,
    partial_variables={
        "format": "JSON 형태로 답변하십시오"
    },
)

print(prompt) # ... partial_variables={'format': 'Return a JSON object.'}
```

2. 체인을 구현하는 문법, LCEL

- LCEL(LangChain Expression Language) 문법 이해

6. 출력 파서(output parser)

1) CommaSeparatedListOutputParser

- 답변을 리스트 형태의 콤마로 구분된 값(CSV)으로 파싱할 때 사용: ['지수', '제니', '로제', '리사']
- 코드 작성 순서

1. 클래스 import

```
from langchain_core.output_parsers import CommaSeparatedListOutputParser

# 파서 초기화
output_parser = CommaSeparatedListOutputParser()
```

2. 프롬프트 불러오기

```
format_instructions = output_parser.get_format_instructions()

print(format_instructions)
```

Your response should be a list of comma separated values, eg: `foo, bar, baz` or `foo,bar,baz` # 실행 결과

- CommaSeparatedListOutputParser의 get_format_instructions() 메서드를 호출하면 미리 작성된 지시문(실행결과)이 반환됨
- 이 지시문을 변수(format_instructions)에 할당

2. 체인을 구현하는 문법, LCEL

- LCEL(LangChain Expression Language) 문법 이해

6. 출력 파서(output parser)

1) CommaSeparatedListOutputParser

- 답변을 리스트 형태의 콤마로 구분된 값(CSV)으로 파싱할 때 사용: ['지수', '제니', '로제', '리사']
- 코드 작성 순서

3. 프롬프트 완성

```
template = """
당신은 K-Pop 정보를 제공하는 AI입니다. 그룹의 멤버 이름을 말해주세요.
K-pop group : {name}

FORMAT :
{format}
"""

prompt = PromptTemplate(
    template=template,
    partial_variables={
        "format": format_instructions
    },
)
```

- 앞서 불러온 지시문을 PromptTemplate의 partial_variables에 전달하여 프롬프트 완성
- 이때 명령어를 전달할 입력 변수({format}) 역시 동일한 이름으로 템플릿에 작성

2. 체인을 구현하는 문법, LCEL

- LCEL(LangChain Expression Language) 문법 이해

6. 출력 파서(output parser)

1) CommaSeparatedListOutputParser

- 답변을 리스트 형태의 콤마로 구분된 값(CSV)으로 파싱할 때 사용: ['지수', '제니', '로제', '리사']
- 코드 작성 순서

4. 모델 생성

```
# model 생성
model = ChatOpenAI(model_name="gpt-5-nano")
```

5. 체인 연결 실행

```
chain = prompt | model | output_parser

result = chain.invoke({"name" : "블랙핑크"})

print('출력 타입 : ', type(result))    # 출력 타입 : <class 'list'>
print('출력 결과 : ', result)          # 출력 결과 : ['지수', '제니', '로제', '리사']
```

- 출력 파서로 output_parser를 연결
- 이 파서가 최종적으로 AIMessage에 있는 content(아직 content는 문자열)를 읽어 리스트로 파싱

2. 체인을 구현하는 문법, LCEL

- LCEL(LangChain Expression Language) 문법 이해

6. 출력 파서(output parser)

2) JsonOutputParser

- 답변을 JSON 객체 형태로 파싱할 때 사용: {'블랙핑크_멤버': ['지수', '제니', '로제', '리사']}
- 코드 작성 순서

1. 클래스 import

```
from langchain_core.output_parsers import JsonOutputParser

# JSON 파서 초기화
output_parser = JsonOutputParser()
```

2. 프롬프트 불러오기

```
format_instructions = output_parser.get_format_instructions()

print(format_instructions)
```

```
Return a JSON object. # 실행 결과
```

- JsonOutputParser의 get_format_instructions() 메서드를 호출하면 미리 작성된 지시문(실행결과)이 반환됨
- 이 지시문을 변수(format_instructions)에 할당

2. 체인을 구현하는 문법, LCEL

● LCEL(LangChain Expression Language) 문법 이해

6. 출력 파서(output parser)

2) JsonOutputParser

- 답변을 JSON 객체 형태로 파싱할 때 사용: {'블랙핑크_멤버': ['지수', '제니', '로제', '리사']}
- 코드 작성 순서

3. 프롬프트 완성

```
template = """
당신은 K-Pop 정보를 제공하는 AI입니다. 그룹의 멤버 이름을 말해주세요.
K-pop group : {name}

FORMAT :
{format}
"""

prompt = PromptTemplate(
    template=template,
    partial_variables={
        "format": format_instructions
    },
)
```

- 앞서 불러온 지시문을 PromptTemplate의 partial_variables에 전달하여 프롬프트 완성
- 이때 명령어를 전달할 입력 변수({format}) 역시 동일한 이름으로 템플릿에 작성

2. 체인을 구현하는 문법, LCEL

- LCEL(LangChain Expression Language) 문법 이해

6. 출력 파서(output parser)

2) JsonOutputParser

- 답변을 JSON 객체 형태로 파싱할 때 사용: {'블랙핑크_멤버': ['지수', '제니', '로제', '리사']}
- 코드 작성 순서

4. 모델 생성

```
# model 생성
model = ChatOpenAI(model_name="gpt-5-nano")
```

5. 체인 연결 실행

```
chain = prompt | model | output_parser

result = chain.invoke({"name" : "블랙핑크"})

print('출력 타입 : ', type(result))    # 출력 타입 : <class 'dict'>
print('출력 결과 : ', result)          # 출력 결과 : 실행할 때마다 JSON 구조가 달라짐
```

- 출력 파서로 output_parser를 연결
- 이 파서가 최종적으로 AIMessage에 있는 content(아직 content는 문자열)를 읽어 리스트로 파싱

2. 체인을 구현하는 문법, LCEL

● LCEL(LangChain Expression Language) 문법 이해

6. 출력 파서(output parser)

2) JsonOutputParser

- 6. 프롬프트 수정
 - 더 상세한 지시를 작성하면 데이터스키마를 유지할 수 있음
 - 하지만 이조차도 완전히 안전한 해결 방법은 아님
 - pydantic 모듈을 활용하여 해결(뒤에서 다룸)

```
template = """
당신은 K-Pop 정보를 제공하는 AI입니다. 그룹의 멤버 이름을 말해주세요.
K-pop group : {name}

FORMAT :
{format}
"""

prompt = PromptTemplate(
    template=template,
    partial_variables={
        "format": """
Return a JSON object like this:
{
    "answer" : "질문에 대한 답변을 작성합니다",
    "source" : "답변 생성에 활용한 출처를 작성합니다."
}
"""
    },
)
chain = prompt | model | output_parser

result = chain.invoke({"name" : "블랙핑크"})

print('출력 결과 : ', result)
```


2. 체인을 구현하는 문법, LCEL

- LCEL(LangChain Expression Language) 문법 이해

- 7. 랭체인 허브(LangChain Hub)

- 프롬프트를 공유하고 활용할 수 있음

- 1) 랭스미스(LangSmith)

- LLM 애플리케이션 개발, 디버깅 및 배포를 위한 도구 제공
 - 요청을 추적하고, 출력을 평가하고, 프롬프트를 테스트하고, 배포를 한 곳에서 관리할 수 있음
 - 랭스미스는 프레임워크에 구매 받지 않으므로 랭체인 오픈 소스 라이브러리를 거치지 않고도 사용할 수 있음
 - 사용자의 프롬프트를 개선해 주는 [hardkothari/prompt-maker](#)를 예시로 사용법을 알아보겠습니다

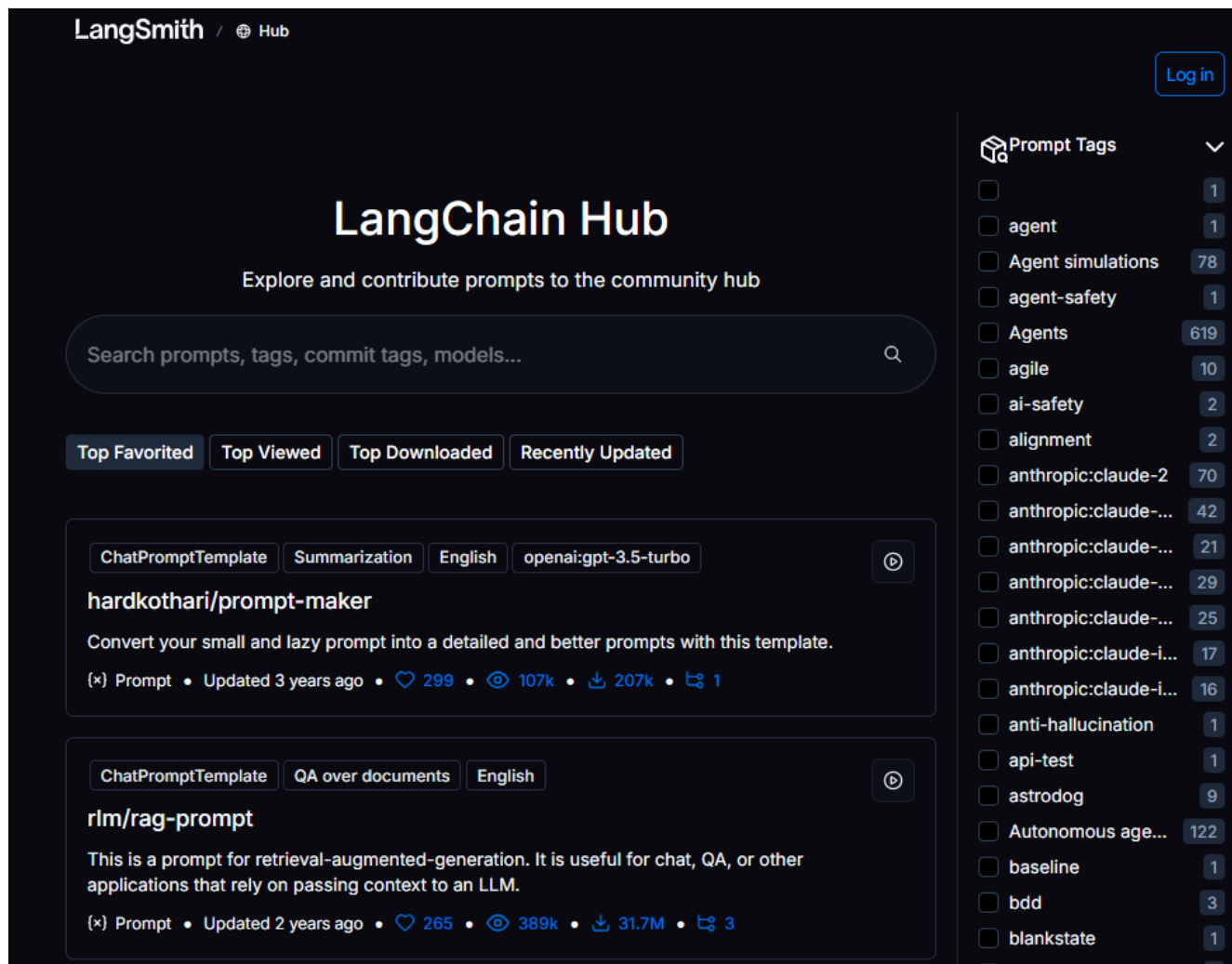
2. 체인을 구현하는 문법, LCEL

- LCEL(LangChain Expression Language) 문법 이해

7. 랭체인 허브(LangChain Hub)

2) 랭스미스(LangSmith) API 키 발급 받기

- <https://smith.langchain.com/hub>
- 회원 가입 후 로그인



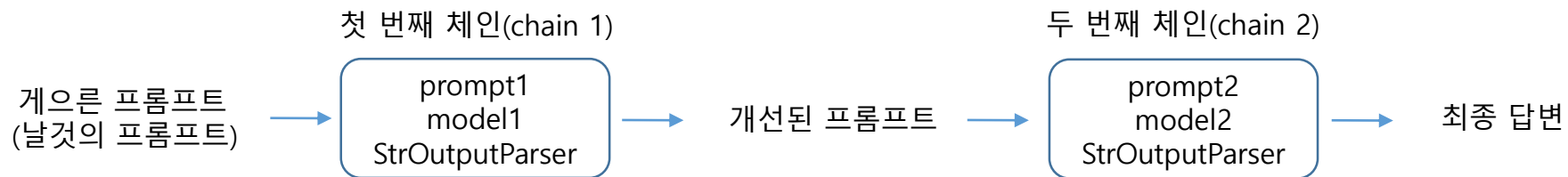
2. 체인을 구현하는 문법, LCEL

● LCEL(LangChain Expression Language) 문법 이해

7. 랭체인 허브(LangChain Hub)

3) 프롬프트 개선

- 사용자의 프롬프트를 개선해 주는 체인 하나를 만들어 프롬프트를 재작성한 뒤, 개선된 프롬프트를 통해 답변을 생성하는 구조를 만든다면 더 높은 수준의 답변을 기대할 수 있을 것임
- 사용자는 평소와 같은 방식으로 서비스를 사용하지만, 사용자 경험(user experience)은 상승
- hardkothari/prompt-maker는 이러한 문제를 해결하기 위해 설계된 프롬프트 중 하나
 - <https://smith.langchain.com/hub/hardkothari/prompt-maker>



- 프롬프트 개선 후 답변 생성

2. 체인을 구현하는 문법, LCEL

- LCEL(LangChain Expression Language) 문법 이해

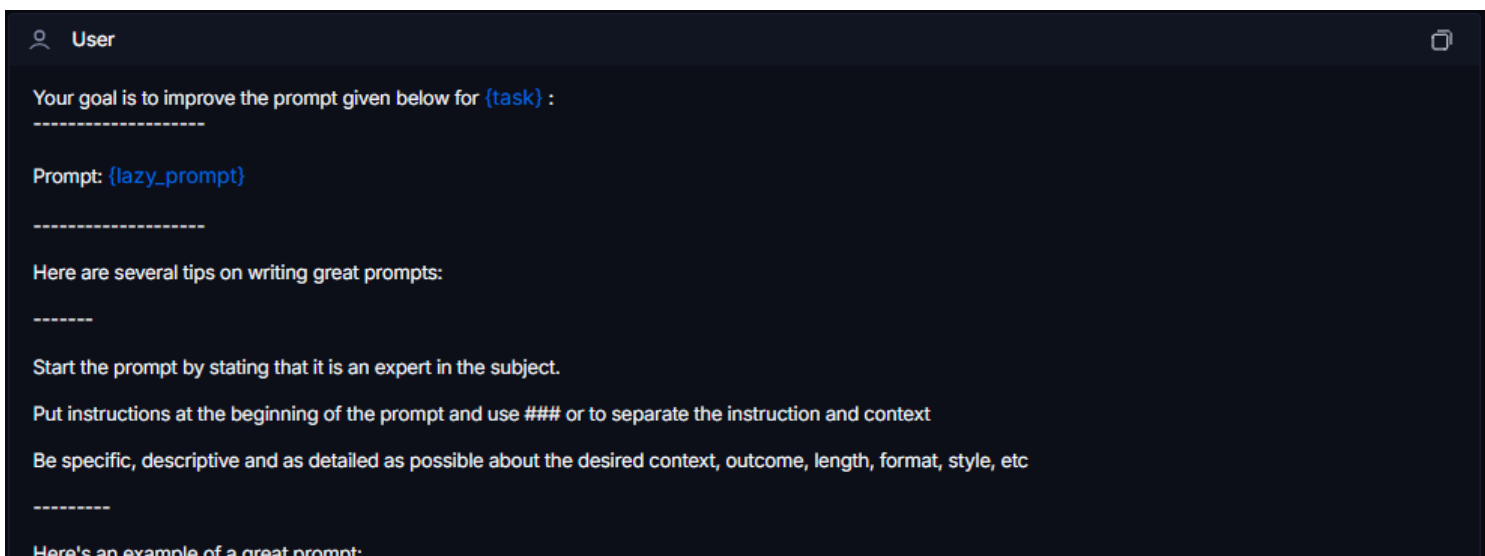
- 7. 랭체인 허브(LangChain Hub)

- 3) 프롬프트 개선

- [hardkothari/prompt-maker](#) 웹 페이지에서 제공되는 예시에는 모델의 퍼소나를 'LLM 모델을 위한 전문 프롬프트 작성자'로 지정해 둠



- User 메시지에서는 {task}와 {lazy_prompt}를 입력 받고 있음. 즉, 진행해야 하는 업무(task)와 사용자가 작성한 기존 프롬프트(lazy_prompt)를 입력 받고 있음



2. 체인을 구현하는 문법, LCEL

- LCEL(LangChain Expression Language) 문법 이해

- 7. 랭체인 허브(LangChain Hub)

- 3) 프롬프트 개선

- hardkothari/prompt-maker를 사용해 사용자의 프롬프트를 개선하는 체인을 만들어 보겠음
 - 사전 정의된 프롬프트 불러오기

```
# Create a LANGSMITH_API_KEY in Settings > API Keys
import os
from dotenv import load_dotenv
from langsmith import Client

load_dotenv()

api_key = os.getenv('LANGSMITH_API_KEY')

client = Client(api_key=api_key)
prompt = client.pull_prompt("hardkothari/prompt-maker", include_model=True)

prompt # 객체 확인
```

2. 체인을 구현하는 문법, LCEL

- LCEL(LangChain Expression Language) 문법 이해

- 7. 랭체인 허브(LangChain Hub)

- 3) 프롬프트 개선

- 체인 구성

```
chain = prompt | ChatOpenAI() | StrOutputParser()

task = """
반드시 한글로 작성되어야 합니다.
사용자의 질문을 읽고, 핵심 키워드를 파악해 전문 지식이 있는 사람의 질문으로 변경해 주세요.
더 체계적이고, 단계적인 질문이 될 수 있도록 변경하세요.
사용자의 질문에서 벗어나서는 안됩니다.
"""

lazy_prompt = "강아지 눈이 좀 이상해요"

improved_prompt = chain.invoke({"task" : task, "lazy_prompt" : lazy_prompt})

print(improved_prompt) # 개선된 프롬프트 확인
```